OpenClaw 核心创新技术架构

基于 external/openclaw 源码与官方文档的系统级深度拆解

研究结论先行

1) OpenClaw 的本质是“可治理 AI 网关”，不是单通道 Bot 。
2）核心壁垒在于：统一控制平面 + 确定性路由 + 分层安全模型。
3）对 DoWhiz 的高价值迁移点：会话键规范、路由层次、运行时策略引擎。

18 页
架构深拆

来源： OpenClaw 源码仓库（v2026.2.23） + docs.openclaw.ai + 社区技术解读

研究方法与结构导航

先

分析方法

- 核心入口： gateway / channels / routing / node-host / media / skills 。
- 证据链：优先本地源码与官方文档；网络资料用于对照视角。
- 输出目标：提炼“可复用架构机制”，而非功能清单。
- 强调工程边界：安全、可运维性、演进成本三维评估。

章节结构

- A. 架构总览与控制平面
- B. 路由 / 会话 / 多代理机制
- C. 插件化通道与流式回复
- D. 安全策略栈与执行审批
- E. 模型容灾、媒体管线与运维治理
- F. 对 DoWhiz 的可迁移路线图

系统分层全景（System of Systems）

外部消息层	WhatsApp / Telegram / Discord / Slack / Signal / iMessage / WebChat
渠道接入层	channels + channel plugins + outbound adapters
路由会话层	bindings tiers + session-key algebra + dmScope
Agent 运行层	pi-agent-core embedded loop + tool streaming
工具执行层	read/write/edit/exec/browser/canvas/nodes
安全治理层	pairing/auth/sandbox/policy/elevated/approvals

锚点： docs/concepts/architecture.md + src/gateway/* + src/channels/* + src/routing/*

创新 1：统一 WebSocket 控制平面

协议内核

关键机制

- 强类型帧： req/res/event （TypeBox schema）。
- 握手先发 connect.challenge，再校验 device signature + nonce。
- operator 与 node 共用同一协议栈，统一 role/scopes/caps 治理。
- side-effect 方法要求 idempotency key，避免重放副作用。
- hello-ok 下发 features/snapshot/policy，天然支持多客户端状态对齐。

握手流程（简化）

① Gateway -> connect.challenge(nonce)
② Client -> req.connect(auth + device + signature)
③ Server：协议版本 / 签名 / 权限 / 配对状态校验
④ res.hello-ok + deviceToken + policy
⑤ 进入 req/res/event 全双工控制面

源码： src/gateway/protocol/schema/frames.ts, src/gateway/server/ws-connection/message-handler.ts

创新 2： 确定性路由 + 会话键代数

1. binding.peer （最精确）
2. binding.peer.parent （线程继承）
3. binding.guild+roles （Discord 角色路由）
4. binding.guild
5. binding.team （Slack）
6. binding.account
7. binding.channel
8. default （兜底代理）

会话键策略

- 主键前缀： agent:<agentId>:...
- dmScope： main / per-peer / per-channel-peer / per-account-channel-peer 。
- 群组键独立： ...:<channel>:group|channel:<id> 。
- identityLinks 可跨渠道归并同一用户身份。

源码： src/routing/resolve-route.ts （8 级优先级） + src/routing/session-key.ts （键构造）

创新 3： 双 Pairing 安全模型

把

DM Pairing （渠道侧）

- 未知发信人先给 pairing code ， 不直接触发模型执行。
- 审批后写入 allowFrom ， 本地文件持久化。
- pairing 请求有 TTL 、数量上限与并发锁。
- 默认策略鼓励 pairing/allowlist ， 而非开放 DM 。

Device Pairing （节点侧）

- 设备连接需 device identity + 签名 + nonce 挑战应答。
- 设备 token 与 role/scopes 绑定，可 rotate/revoke 。
- approve/reject/remove 全量可审计，广播 pair 事件。
- 使“节点能力执行”可控且可追溯。

锚点： docs/channels/pairing.md, src/pairing/pairing-store.ts, src/gateway/server-methods/devices.ts

创新 4： 通道插件化契约（Channel Plugin Contract）

把

ChannelPlugin 典型适配面

- config / setup / pairing / security / groups / outbound / status
- gateway / auth / commands / threading / streaming / messaging
- directory / resolver / actions / heartbeat / agentTools
- 同一通道生命周期在同一个契约中完成，不再散落在多模块“拼装”。
- 配合 plugins allow/deny 与 schema 校验，实现扩展能力和治理能力同步增长。

源码： src/channels/plugins/types.plugin.ts + types.adapters.ts + docs/tools/plugin.md

创新 5： 双层流式回复机制

B 兼顾可读

Block Streaming （真实消息块）

- 按 text_end 或 message_end 刷出块消息。
- min/max 字符阈值 + break preference （段落 / 换行 / 句子）。
- 代码块分割保护：必要时自动闭合与重开 fence 。
- 可配置 coalesce ，避免碎片化“连发刷屏”。

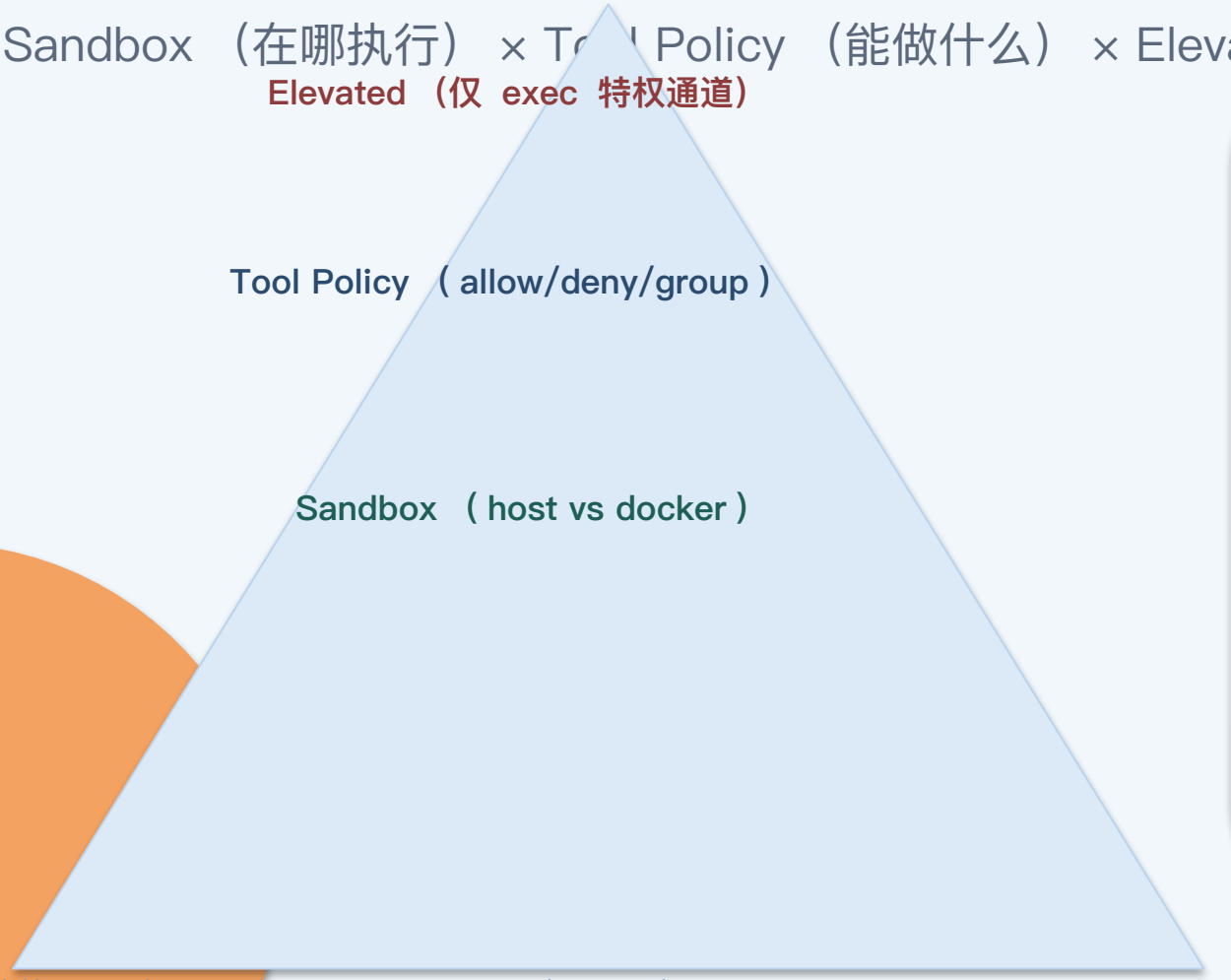
Preview Streaming （临时预览）

- Telegram/Discord/Slack 支持预览消息编辑。
- 模式： off / partial / block / progress 。
- 与 block streaming 显式避冲突，防止“双重流式”。
- 核心是交互体验层，不替代最终消息持久语义。

文档： docs/concepts/streaming.md

创新 6： 运行时安全三层栈

Sandbox （在哪执行） × Tool Policy （能做什么） × Elevated （仅 exec 特权通道）



工程价值

- 把“执行位置”和“权限策略”解耦，避免单开关幻觉。
- 可按 agent/session 维度差异化配置。
- 对高风险工具默认可收敛到 deny + 审批。
- 结合 pairing 与 scope ，可形成端到端最小权限路径。

文档： docs/gateway/sandboxing.md + docs/gateway/sandbox-vs-tool-policy-vs-elevated.md

创新 7： system.run 策略引擎（非布尔开关）

命令执行由 security/ask/allowlist/approval 联合求值

命令解析
argv/shell

allowlist 分析
匹配与风险判定

审批策略
ask/off/on-miss

执行通道
host 或 companion

事件回传
exec.finished

关键点

- 可识别 shell wrapper （如 bash -c / cmd.exe /c ）并阻断绕过式调用。
- allow-once / allow-always 与 allowlist 记录联动，形成可追踪演进策略。
- 配合 exec approval 事件，支持“运行中人类确认”闭环。

源码： src/node-host/exec-policy.ts + src/node-host/invoke-system-run.ts

创新 8： 模型容灾 = profile 轮转 + fallback 链

Provider 内部

- auth profiles （OAuth/API key）按规则轮转。
- 对 rate-limit/auth/billing 建立 cooldown/disable 状态。
- 会话维度 profile stickiness，优先稳定缓存与上下文连续性。
- 失败后优先 provider 内切换，再决定是否跨模型 fallback。

跨模型 fallback

- agents.defaults.model.primary + fallbacks 按序尝试。
- 当 provider 内 profile 已耗尽，切换到下一个模型。
- 兼顾可靠性与成本，不把失败处理留给外层调用方。
- 同一机制可支撑多 provider 统一接入。

文档： docs/concepts/model-failover.md

创新 9： OpenAI / OpenResponses 兼容网关

外

入口

- POST /v1/chat/completions （OpenAI 兼容）
- POST /v1/responses （OpenResponses 兼容，支持 item/tool/image/file 流程）
- 与 Gateway auth 、 agent routing 、 sessionKey 规则保持一致
- 请求在内部仍走统一 agentCommand 链路，减少实现分叉
- stream 模式统一映射为 SSE 事件输出

源码： src/gateway/openai-http.ts + openresponses-http.ts ；文档： docs/gateway/*http-api.md

创新 10： Skills 平台（分层加载 + 动态刷新）

把“能力提示词工程”升级为可配置、可门控、可热更

workspace/skills （最高优先级）

~/.openclaw/skills （ managed ）

bundled skills （内置）

extraDirs + plugin skills

机制亮点

- metadata gate： bins/env/config/os 条件过滤。
- session 技能快照复用，降低每轮重建开销。
- watcher 监听 SKILL.md 变化，下一轮自动刷新。
- 可与远端 node 能力联动，动态改变技能可用集合。

文档： docs/tools/skills.md ；源码： src/agents/skills/workspace.ts + refresh.ts

创新 11： 媒体管线的安全与生命周期治理

从

重点： OpenClaw 把媒体当作“潜在攻击输入”处理，而非普通附件。

URL 输入 → SSRF Guard DNS/IP/ 重定向 → 响应限流 maxBytes → MIME 识别 magic/header/ext → 安全落盘 path/symlink 防护 → TTL 清理 单次访问回收

源码： src/media/fetch.ts + store.ts + server.ts

可靠性与运维：把长期运行作为一等公民

Session 维护

- pruneAfter / maxEntries / rotateBytes / maxDiskBytes 。
- cleanup 支持 warn 与 enforce 模式。
- 避免会话索引与转录文件无限膨胀。

Context 治理

- compaction： 持久化摘要进入 JSONL 。
- session pruning： 请求前内存裁剪 toolResult 。
- 二者分工明确，避免上下文失控。

传输健壮性

- provider retry policy （按渠道定制）。
- auth/hook 限流与失败回退。
- idempotency + dedupe 定时清理。

文档： docs/concepts/session.md / compaction.md / session-pruning.md / retry.md

工程规模证据（external/openclaw 本地统计）

不是原型级项目，而是大规模持续演进的生产工程

3676

src TypeScript 文件

1366

src 测试文件（*test.ts）

652

docs 文档文件

28MB

src 体量

工程启示

- 复杂度足够高，架构设计重点应放在治理能力与演进稳定性，而非单次功能实现。
- 高测试密度意味着其核心机制（路由、安全、协议）具备较强回归保障基础。

统计命令：rg --files + wc/du （本地执行）

面向 DoWhiz 的迁移路线图（建议）

先

Phase 1 （2–4 周）

会话与路由基建

- 定义 session key 规范
- 固化 binding tiers
- 补回归测试矩阵

Phase 2 （4–8 周）

执行与安全治理

- 引入 sandbox/tool policy 分层
- 建设 exec 审批链路
- 分离 DM 与 device pairing

Phase 3 （持续）

生态与兼容层

- 插件化通道契约
- 标准化 HTTP 兼容入口
- skills 与能力目录化

迁移原则：先语义一致性（路由 / 会话），再能力扩展（插件 / 执行）。

结论： OpenClaw 的竞争力来自“系统治理能力”

统一控制面、确定性路由、分层安全策略共同构成其技术护城河

最终判断

- OpenClaw 已从“聊天机器人”进化为“个人 AI 网关操作系统”。
- 其创新重心在架构语义（协议 / 路由 / 会话 / 策略），不是单一模型能力。
- 对 DoWhiz 的最优学习路径：优先迁移治理框架，再做功能扩展。
- 这能在复杂度上升时保持可维护、可审计、可持续迭代。

谢谢。附：详细技术文档 openclaw_ 核心创新技术架构分析 .md